MENTOR CONNECT

Intelligent Student–Mentor Matching and Guidance Platform

College Expo Project Report | Medium-Level | Zero-Cost Prototype

Project Slogan: Describe → Understand → Match → Connect → Mentor → Improve

1. Abstract

College students frequently need help with projects, programming, research, placements, technologies and academic decisions, but they often do not know which faculty member, senior student or alumnus is the right person to approach. Mentor Connect is an intelligent mentoring platform that converts a student's natural-language problem into a structured requirement, identifies relevant skills and skill gaps, checks mentor experience and availability, calculates a compatibility score, and recommends the top three mentors.

Unlike a simple mentor directory, the system performs problem-based matching. A student can describe a problem such as “I am struggling with React and Firebase in my final-year project” rather than manually selecting a skill. The platform extracts React, Firebase, web development, project guidance and debugging-related requirements and uses them to rank mentors. It also considers mentor workload, previous feedback and availability.

The prototype can be implemented entirely with open-source technologies such as React, Tailwind CSS, Python/FastAPI, SQLite, spaCy/scikit-learn and local rule-based NLP, without paid AI APIs.

2. Problem Statement

Students commonly face a mentor-discovery problem: the correct source of guidance exists inside the college ecosystem, but the student may not know whom to contact. Existing approaches such as asking classmates, searching faculty lists or messaging seniors are manual and do not systematically compare skills, experience, availability and mentoring effectiveness.

A system is therefore required that accepts a student's problem in natural language, understands the requirement, identifies suitable mentors, checks availability and workload, provides transparent compatibility scores, and supports a complete request-to-feedback mentoring workflow.

3. Proposed Solution

Mentor Connect provides a centralized platform containing faculty, senior-student and alumni mentors. A student describes what they need help with. The system performs requirement analysis and skill extraction, detects the student's level and possible skill gaps, searches the mentor database, calculates a compatibility score, checks availability and overload, and presents the top three recommendations with explanations.

If the selected mentor rejects the request or becomes unavailable, the system automatically re-matches the student with the next suitable mentor. After a session, feedback is stored and used to improve mentor effectiveness scoring.

4. Objectives

- Centralize access to faculty, senior and alumni mentors.
- Allow students to describe problems naturally rather than searching manually.
- Extract skills, domain, requirement type and student level from the problem description.
- Match students using skill, experience, domain, availability and feedback.
- Detect skill gaps and recommend targeted guidance.
- Prevent mentor overload from concentrating requests on a few popular mentors.
- Provide transparent compatibility scores and reasons.
- Support session requests, acceptance, mentoring and feedback.
- Automatically re-match when a mentor is unavailable.
- Build the complete expo prototype using zero-cost/open-source technologies.

5. Target Users

User	Main Needs
Students	Find the right mentor, request sessions, view recommendations and receive skill-gap guidance.
Faculty Mentors	Provide academic, project, research and subject guidance.
Senior Mentors	Help with coding, projects, placements and technologies.
Alumni Mentors	Provide industry, internship, career and professional guidance.
Administrators	Manage mentors, skills, availability, requests and platform analytics.

6. Existing System and Limitations

Existing Approach	Limitation
Ask classmates/seniors	Depends on personal network and may identify the wrong mentor.
Faculty directory	Shows names but does not intelligently match problems to expertise.
Generic search	Requires the student to know the correct skill keywords.
Manual appointment process	Does not automatically consider availability or workload.
Popularity-based selection	Can overload the same mentor and ignore effectiveness.

Mentor Connect improves this by making matching problem-based, compatibility-driven and availability-aware.

7. Major Features

- **Natural-Language Student Requirement:** Students describe their problem in ordinary language.
- **Skill Extraction:** Identifies technologies, subjects, domains and task types.
- **Problem-Based Matching:** Matches the actual problem rather than only a selected skill.
- **Student-Level Detection:** Estimates beginner, intermediate or advanced requirements.
- **Mentor Profiles:** Stores skills, experience, projects, internships, ratings and availability.
- **Smart Compatibility Score:** Combines multiple matching factors into a 0–100 score.
- **Availability Matching:** Favors mentors whose available slots overlap the student's preferred time.
- **Peer + Faculty + Alumni Mentors:** Broadens the mentoring ecosystem.
- **Mentor Overload Detection:** Penalizes or flags mentors with excessive active requests.
- **Automatic Re-Matching:** Finds the next best mentor after rejection/unavailability.
- **Feedback-Based Improvement:** Updates mentor effectiveness using post-session ratings.
- **Skill Gap Detection:** Identifies missing prerequisites and recommends targeted mentoring.
- **Transparent Explanations:** Shows why a mentor received a particular score.
- **Session Request Workflow:** Supports request, acceptance, mentoring and feedback.

8. Problem-Based Requirement Analysis

Example input: “My Android application crashes when I use Firebase authentication.”

The system can convert the sentence into a structured requirement:

Extracted Element	Detected Value
Domain	Mobile / Android
Skills	Android, Firebase
Topic	Authentication
Requirement	Debugging
Likely Level	Intermediate
Preferred Mentor Type	Senior / Faculty with project experience

This structured representation is then used by the matching engine.

9. Skill Gap Detection

The platform can compare the student's known skills with the skills required for the requested goal. For example, a student requesting an ML project may have Python and Pandas but lack statistics and ML fundamentals. The system can flag those gaps and recommend a mentor who specifically covers them.

Skill	Student Status
Python	✓ Known
Pandas	✓ Known
Statistics	■ Needs improvement
ML Fundamentals	✗ Gap
Recommended Action	Mentor guidance for Statistics + ML basics

10. Mentor Profile

Example — Karthik
Final Year CSE

Skills: Python, Machine Learning, Data Science, TensorFlow
Experience: 3 projects, 2 internships
Mentoring: 18 students
Rating: 4.7/5
Availability: Monday, Wednesday, Friday — 6 PM to 8 PM

11. Smart Mentor Matching

The matching engine ranks mentors rather than simply listing them. A practical prototype formula is:

Compatibility Score = Skill Match + Experience Match + Availability Match + Domain Match + Feedback/Effectiveness + Load Adjustment

The individual factors can be normalized and weighted to produce a final score between 0 and 100. Example weights:

Factor	Suggested Weight
Skill Match	40%
Experience Match	20%
Availability Match	15%
Domain/Project Match	15%
Feedback/Effectiveness	10%

A mentor overload penalty can then reduce the final score when the mentor has too many active students or pending requests.

12. Example Compatibility Calculation

Factor	Score
Skill	40 / 40
Experience	18 / 20
Availability	15 / 15
Domain	14 / 15
Rating/Effectiveness	8 / 10
Total	95 / 100

The user interface should show both the score and the explanation, for example: “95% match — strong ML skill match, relevant project experience, available during your preferred time, and high mentoring effectiveness.”

13. Availability Matching

Availability is treated as a matching factor rather than an afterthought. If a student is available from 7–9 PM, a mentor available from 7–8 PM receives a strong availability match, while mentors available only from 5–6 PM or 9–10 PM are ranked lower.

This can be implemented using time-interval overlap calculations in Python.

14. Mentor Overload Detection

A popular mentor may otherwise receive nearly every request. Mentor Connect monitors active students, pending requests and capacity. For example, if Mentor A has 18 active students and 7 pending requests, the system can display **“Mentor overloaded”** and recommend another mentor with similar skills and better capacity.

This feature improves fairness, response time and overall mentor utilization.

15. Automatic Re-Matching

When a mentor rejects a request, becomes unavailable or reaches capacity, the system does not force the student to restart the search. The matching engine excludes the unavailable mentor, recalculates candidates and recommends the next suitable mentor.

16. Feedback-Based Improvement

After a mentoring session, the student can rate usefulness. The system stores feedback and calculates an effectiveness indicator in addition to raw popularity. Over time, mentors who consistently provide useful guidance can receive a stronger ranking for relevant requirements.

17. Complete System Workflow

STUDENT
↓
Describe the Problem
↓
Requirement Analysis
↓
Skill / Domain / Level Extraction
↓
Skill Gap Detection
↓
Mentor Database
↓
Matching Algorithm
↓
Availability + Workload Check
↓
Compatibility Score
↓
Top 3 Mentors
↓
Student Selection
↓
Session Request
↓
Mentor Accept / Reject
↓
Mentoring Session
↓
Feedback
↓
Mentor Effectiveness Updated

18. System Architecture

Layer	Components
Presentation	React, Tailwind CSS, student dashboard, mentor dashboard, admin screens
API / Backend	Python + FastAPI, authentication, request/session APIs
Intelligence	Skill extraction, requirement classification, matching, availability, overload, skill-gap and recommendation logic
Data	SQLite database containing users, mentors, skills, availability, requests, sessions and feedback
Visualization	Recharts or Chart.js for match and workload analytics

Architecture Flow: Student → React UI → FastAPI → Requirement/NLP Engine → Mentor Matching Engine → Availability & Load Engine → Recommendation → Database → Dashboard.

19. Technology Stack

Component	Technology	Purpose
Frontend	React	Interactive web interface

Component	Technology	Purpose
Styling	Tailwind CSS	Responsive UI
Backend	Python + FastAPI	REST APIs and business logic
Database	SQLite	Zero-cost local data storage
NLP	spaCy / NLTK + Python rules	Skill and requirement extraction
Similarity	scikit-learn	Text/skill similarity
Charts	Recharts / Chart.js	Analytics and dashboards
Authentication	Local JWT	Prototype login/session security
Development	VS Code + Git + GitHub	Development and version control

Core software/API cost: ■0 for the prototype when using local/open-source components.

20. Database Design

Table	Important Fields
Users	user_id, name, email, department, semester, role
Mentors	mentor_id, user_id, mentor_type, experience_years, bio, capacity, rating
Skills	skill_id, skill_name, domain
MentorSkills	mentor_id, skill_id, proficiency
Availability	availability_id, mentor_id, day, start_time, end_time
StudentSkills	student_id, skill_id, proficiency
Requests	request_id, student_id, mentor_id, problem_text, status, score, created_at
Sessions	session_id, request_id, scheduled_at, duration, status
Feedback	feedback_id, session_id, rating, usefulness, comments
SkillGaps	gap_id, student_id, skill_id, severity

21. Core Algorithm

- Receive the student's natural-language problem.
- Normalize and clean the input text.
- Extract technologies, subjects, domains and task type.
- Estimate student level and identify required experience.
- Compare requested skills against mentor skills.
- Calculate project/domain and experience relevance.
- Check student–mentor availability overlap.
- Calculate mentor workload and apply overload penalties.
- Include historical feedback/effectiveness.
- Calculate final compatibility score.
- Rank all eligible mentors.
- Return the top three mentors with explanations.
- Create a session request for the selected mentor.
- Re-match automatically if the mentor is unavailable or rejects.
- After the session, store feedback and update effectiveness.

22. Matching Pseudocode

```
INPUT: student_problem, student_profile, preferred_time
1. requirements = extract_requirements(student_problem)
2. gaps = detect_skill_gaps(student_profile, requirements)
3. mentors = get_available_mentors()
4. for mentor in mentors:
skill = calculate_skill_match(requirements, mentor.skills)
exp = calculate_experience_match(requirements, mentor.experience)
time = calculate_availability_match(preferred_time, mentor.availability)
domain = calculate_domain_match(requirements, mentor.domain)
feedback = calculate_effectiveness(mentor.feedback)
load = calculate_load_penalty(mentor.active_students, mentor.pending_requests)
score = weighted_sum(skill, exp, time, domain, feedback) - load
5. rank mentors by score
6. return top 3 mentors + explanations + skill gaps
```

23. End-to-End Expo Demo

Step	Demo Action	System Result
1	Student enters: “I need help with my React + Firebase project.”	Problem is analyzed.
2	System displays extracted information	React ✓, Firebase ✓, Web Project ✓, Debugging/Project Guidance.
3	Matching engine runs	Top mentors ranked.
4	Results shown	Arun 94%, Priya 87%, Karthik 81%.
5	Student selects Arun	Available today at 7:00 PM.
6	Request submitted	Mentor dashboard receives request.
7	Mentor accepts	Session is confirmed.

Step	Demo Action	System Result
8	Session completed	Student gives 5/5 feedback.
9	Feedback stored	Mentor effectiveness score is updated.

24. Suggested UI Screens

- Student Login / Registration
- Student Home Dashboard
- Problem Description Input
- Detected Skills and Requirement Review
- Skill-Gap Panel
- Top 3 Mentor Recommendations
- Mentor Profile and Compatibility Explanation
- Availability / Session Booking
- Student Request History
- Mentor Dashboard
- Pending Request Screen
- Session Management
- Feedback Screen
- Admin Mentor and Skill Management
- Workload and Matching Analytics

25. Advantages

- Reduces the time needed to find an appropriate mentor.
- Works with natural-language problem descriptions.
- Supports faculty, senior and alumni mentors.
- Considers availability rather than only expertise.
- Reduces mentor overload.
- Provides transparent matching explanations.
- Supports automatic re-matching.
- Uses feedback to improve mentor ranking.
- Detects skill gaps and prerequisite needs.
- Can be developed and demonstrated at zero software/API cost.

26. Limitations

- NLP extraction may be imperfect for ambiguous student descriptions.
- Skill names and mentor profiles must be maintained accurately.
- Availability data must be kept up to date.
- Feedback can be subjective and should not be treated as an absolute measure of mentor quality.

- Real-time college directory or messaging integration may require institutional permissions.
- The first prototype should allow students and administrators to correct extracted requirements.

27. Security and Privacy

- Use authenticated access for student and mentor dashboards.
- Store only information required for matching and session management.
- Separate student data so users cannot access another student's private information.
- Use password hashing and signed local JWT tokens for the prototype.
- Validate API inputs and restrict administrative operations.
- Keep the zero-cost prototype local where possible.
- Do not expose private student problems or feedback publicly.

28. Future Scope

- College ERP and student-directory integration.
- Google Calendar synchronization.
- Automated email/notification support.
- Local LLM support for richer problem understanding.
- Multilingual English/Tamil/Hindi problem analysis.
- Video meeting integration.
- Personalized learning-roadmap generation.
- Mentor analytics and institutional dashboards.
- AI-assisted session summaries.
- Predictive mentor recommendation based on historical outcomes.
- Mobile application for Android/iOS.

29. Innovation / USP

Mentor Connect is differentiated from a normal mentor directory because it treats mentoring as an intelligent matching problem. Its strongest innovations are problem-based matching, transparent compatibility scoring, availability-aware recommendations, mentor overload detection, automatic re-matching, feedback-based effectiveness and skill-gap detection.

The system's key question is not “Who are the mentors?” but “Given this student's exact problem, who is the best available mentor right now?”

30. MVP for College Expo

Priority	Features
Must Have	Student login, mentor profiles, natural-language problem input, skill extraction, matching score, top 3 recommendations, availa
Nice to Have	Skill-gap detection, mentor overload visualization, automatic re-matching, charts, admin dashboard.
Future	Calendar integration, video meetings, multilingual NLP, local LLM, mobile application.

Recommended MVP flow: **Describe** → **Extract** → **Match** → **Check Availability** → **Recommend** → **Request** → **Accept** → **Feedback**.

31. Suggested Team and Timeline

Member	Responsibility
Member 1	React frontend, Tailwind UI and student dashboard
Member 2	FastAPI backend, SQLite database and authentication
Member 3	NLP, skill extraction, matching, availability and overload algorithms
Member 4	Testing, UI polish, documentation, demo and presentation

A polished expo prototype can be targeted for approximately 2–4 weeks, depending on team size and feature scope.

32. Conclusion

Mentor Connect addresses a common college problem: students need guidance but often do not know which person is best suited to help them. By converting natural-language problems into structured requirements and intelligently comparing mentor expertise, experience, availability, workload and feedback, the system provides a practical and demonstrable mentoring solution.

Its value goes beyond a directory or booking application. Mentor Connect is a decision-support platform that recommends the most suitable mentor, explains the match, identifies skill gaps, avoids mentor overload and continuously improves through feedback. With React, FastAPI, SQLite and open-source NLP libraries, the project is feasible as a medium-level college expo prototype with a zero-cost technology stack.

33. Key Demo Screen

Mentor	Match	Availability	Status
Arun	94%	Today 7:00 PM	Recommended
Priya	87%	Tomorrow 6:00 PM	Alternative
Karthik	81%	Wed 7:30 PM	Alternative

Recommended Mentor: **Arun — 94%**

Why: React ✓ Firebase ✓ Web project experience ✓ Relevant mentoring feedback ✓ Preferred time available ✓ Workload within capacity ✓

Project slogan: Describe → Understand → Match → Connect → Mentor → Improve